AgriScan API Feature Reference

Endpoints, contracts and implementation status

AgriScan monorepo — `apps/api`

Version 1.0 — October 2026

Contents

1 How to read this document **2**

2 System shape at a glance **2**

3 Feature catalogue **3**

3.1 F1 — Health check . . . 3

3.2 F2 — Registration & session (auth) . . . 3

3.3 F3 — Crop scan pipeline (core product) . . . 4

3.4 F4 — AgriBot chat (RAG + LLM) . . . 4

3.5 F5 — Market intelligence (KAMIS + directories) . . . 5

3.6 F6 — Inputs marketplace (agencies) . . . 5

3.7 F7 — Wallet & M-Pesa top-up . . . 5

3.8 F8 — Farm Health Score & lending API . . . 6

3.9 F9 — Admin (moderation & revenue) . . . 6

3.10 F10 — USSD channel . . . 7

3.11 F11 — Legacy server-rendered routes . . . 7

4 Access matrix **7**

5 Appendix — Endpoint index **8**

1 How to read this document

Every API feature in the AgriScan monorepo (`apps/api` — Django 5 + DRF + SimpleJWT), with its endpoints, request/response contracts, business rules and current implementation status.

Status legend

- **Implemented** — endpoint exists, is wired to the frontend, and works end-to-end in the current build.
- **Partial** — code exists but has a known gap (no live credentials, duplicate logic, unvalidated, or dead route).
- **Planned** — described in the roadmap/docs, not built.
- **Legacy** — old server-rendered Django view kept for backward compatibility; superseded by a JSON endpoint.

Base URLs

- API base (JSON): `http://localhost:8000/api/v1/` in dev, `https://api.agriscan.co.ke/api/v1/` in production (behind nginx).
- Web app: `http://localhost:3000` (Next.js), reads the base from `NEXT_PUBLIC_API_URL`.
- A few non-prefixed routes live at the site root (M-Pesa callback, legacy HTML views, allauth) — flagged inline below.

2 System shape at a glance

Layer	What it is
Frontend	Next.js 15 App Router + TanStack Query, wrapped by Capacitor (Android). The browser calls Django directly (CORS); there is no Next API layer.
API	Django 5 + DRF, Gunicorn (2 workers), JWT auth, scoped throttles.
Async work	Celery with a *single* task: `inference.run_scan_job` on queue `scan`. Chat/RAG run inline in the request.
ML	TensorFlow: EfficientNet gatekeeper + per-crop MobileNetV2 (`.h5`), loaded in the *worker*, never in the API container.
Data	PostgreSQL (source of truth), Redis (cache + Celery broker), SQLite cache for KAMIS prices, Chroma/BM25 for RAG.
Payments	Safaricom Daraja STK Push; wallet ledger on `Wallet`.
Channels	Web portal, Android wrap, USSD (simulator live, provider webhook ready).

Authentication model

- **SPA (browser/app):** Bearer JWT. Access token `agriscan_access` + refresh `agriscan_refresh` in `localStorage`; 30-minute idle timeout enforced both client and server (`IdleTimeoutJWTAuthentication`).
- **Machine-to-machine (lenders):** `Authorization: Bearer <LendingAPIKey raw secret>`, hashed at rest; a legacy plaintext `PartnerAPIKey` is still accepted while `ALLOW_LEGACY_PARTNER_KEYS` or `DEV_ENV` is on.
- **Role resolution** is implicit: the profile relation a user owns (`farmer_profile`, `buyer_profile`, `agency_profile`, `lending_partner_profile`) decides the role; `is_staff` maps to admin. Custom permissions: `IsFarmer`, `IsBuyer`, `IsAgency`, `IsLendingPartner`, `IsApprovedLendingPartner`, `IsStaffUser` — all in `accounts/permissions.py`.

Throttle scopes (production rates)

All scopes are relaxed while `DEV_ENV=True` (that flag also disables throttling entirely and allows all CORS origins).

Scope	Prod rate	Used by
`auth`	10/min	login, refresh, register
`scans`	5/min	`POST /scans/`
`chat`	5/min	AgriBot
`payments`	3/min	M-Pesa top-up
`weather`	10/min	weather lookup
`ussd`	30/min	gateway + simulator
`lending_score`	30/min	score lookups
`anon` / `user`	60 / 120 per min	global DRF defaults

3 Feature catalogue

Tables use: **Method** · **Path** · **Access**. All paths are relative to `/api/v1/` unless marked "root".

3.1 F1 — Health check

Method & path	Purpose	Access
`GET /health/`	Postgres + Redis liveness probe	Anonymous

Status: **Implemented**

Response:

```
{"status": "ok" | "degraded",
 "postgres": true, "redis": true, "dev_env": false}
```

Returns `503` when Postgres is unreachable, `200` otherwise (a Redis-only outage reports `"degraded"` with HTTP 200).

3.2 F2 — Registration & session (auth)

Method & path	Purpose	Access
`POST /auth/register/farmer/`	farmer signup	anon, `auth` throttle
`POST /auth/register/buyer/`	buyer signup (multipart: photo)	anon
`POST /auth/register/agency/`	input-agency signup	anon
`POST /auth/register/lender/`	lender signup	anon
`POST /auth/login/`	JWT pair; email *or* username	anon
`POST /auth/refresh/`	rotate access token (idle-aware)	anon
`POST /auth/logout/`	blacklist refresh token	authed
`GET /me/`	current user + role + profile + wallet	authed
`PATCH /me/`	update profile fields	authed
`GET /weather/`	OpenWeather for farmer/buyer location	farmer/buyer

Status: **Implemented** (Google OAuth also available via django-allauth at root `/accounts/`).

Registration contract: common fields `email` (required), `password` (min 8), optional `username`/ `first_name`/`last_name`; role extras — farmer: `phone_number`, `county`; buyer: `phone_number`,

`county`, `photo` (image, required); agency: `agency_name`, `agency_type`, `phone`, `county`; lender: `organization_name`, `organization_type`, `registration_number`, `contact_phone`, `county`.

Returns `201` with `{access, refresh, role, user}`; the client then calls `GET /me/` for the full profile.

`GET /me/` payload shape:

```
{"user":  {...}, "role": "farmer",
 "profile": {..., "health_score", "risk_tier",
             "opt_in_for_lending"},
 "wallet": {"balance": "0.00",
            "scan_count": 0, "chat_count": 0}}
```

(`wallet` only present for farmers; `null` otherwise.)

Profile updates via `PATCH /me/`: names, email, phone, county, country, products, farm fields, profile photo (or `clear_profile_photo`), agency/lender organisation fields, and the `opt_in_for_lending` consent flag.

3.3 F3 — Crop scan pipeline (core product)

Method & path	Purpose	Access
`GET /crops/`	crop catalogue + disease classes	farmer
`POST /scans/`	upload image, enqueue Celery job	farmer, `scans` throttle
`GET /scans/`	last 50 jobs for the user	farmer
`GET /scans/<job_id>/`	poll job status/result + wallet balance	farmer

Status: **Implemented** (async, queue-guarded).

Business rules:

- Image must be an image, ≤ 5 MB.
- **2 free scans** (`wallet.scan_count < 2`); after that each scan costs **KSh 20** — `402 Payment Required` with `{required, balance}` when the wallet is short.
- Global queue cap `MAX_QUEUED_SCAN_JOBS=20` → `503` + `Retry-After: 30`.
- Per-user cap `MAX_QUEUED_SCANS_PER_USER=3` → `429` + `Retry-After`.
- Enqueue failure → job marked `failed`, image cleaned up, `503` returned.
- Job statuses: `queued` → `processing` → `completed | rejected | failed`.

Result payload (on `GET /scans/<id>/`): `result_label`, `confidence`, `report` (JSON: overview, recommendations, prevention, severity, etc.), `gatekeeper_passed`, `error_message`, timestamps, plus live `wallet_balance`.

`GET /crops/` returns every known crop with its disease class list and a `supported` flag (membership in `SUPPORTED_SCAN_CROPS` — 18 crops listed; only 11 `.h5` weights are committed, the rest require `manage.py sync_models`).

3.4 F4 — AgriBot chat (RAG + LLM)

Method & path	Purpose	Access
`POST /chat/messages/`	ask AgriBot, get structured advice	farmer, `chat` throttle

Status: **Implemented** (English + Kiswahili). RAG, weather and LLM calls all run *inside* the request (no Celery), so latency and failures are user-visible.

Flow: strip HTML from `message` → check wallet (**5 free chats**, then **KSh 10** each; `402` with `needs_topup:true` if short) → build context (farmer name, county, live weather, RAG snippets capped at 4500 chars) → `ChatGroq` at temperature 0.3 → parse JSON.

Response shape:

```
{"response": {"overview", "diagnosis",
              "recommendations": [...],
              "prevention", "weather_note", "local_tip"},
 "wallet_balance": "90.00", "chat_count": 6,
 "farmer": {"name": "...", "location": "Nakuru, Kenya"}}
```

On LLM/JSON-parse failure the API still returns `200` with a degraded `response` object (empty `diagnosis`, error text folded into `local_tip`).

3.5 F5 — Market intelligence (KAMIS + directories)

Method & path	Purpose	Access
`GET /market/kamis-prices/?crop=`	price lookup from local SQLite cache	farmer/buyer/staff
`GET /market/buyers/?county=`	buyer directory (phone/email masked)	farmer/staff
`GET /market/farmers/?county=&crop=`	farmer directory for buyers	buyer/staff

Status: **Implemented** (offline KAMIS cache; no live scrape). Prices come from `apps/api/data/kamis_prices.db` refreshed by a manual/cron ingest.

Contract notes:

- `crop` is required; missing → `400`.
- Empty cache → `503` with a `detail` explaining the cache is empty.
- Directories return masked contact details (`_mask_phone`, `_mask_email`) plus a `verified` / `verification_badge` flag.
- Legacy HTML twins (`/market_firms/`, `/farmer_data/`) still serve *unmasked* contact data — see F11.

3.6 F6 — Inputs marketplace (agencies)

Method & path	Purpose	Access
`GET /inputs/agencies/`	list agencies (`county`, `type`, `q`)	farmer
`GET /inputs/products/`	available products (`county`, `category`, `agency_id`, `q`)	farmer
`GET /inputs/my-products/`	own catalogue	agency
`POST /inputs/my-products/`	create item (multipart or JSON)	agency
`PATCH /inputs/my-products/<id>/`	partial update	agency
`DELETE /inputs/my-products/<id>/`	remove item	agency

Status: **Implemented**. Ownership is enforced server-side (`_get_owned` → `404` for someone else's product, so cross-tenant edits return `404`, not `403`). Only `is_available=true` products appear in public listings.

3.7 F7 — Wallet & M-Pesa top-up

Method & path	Purpose	Access
`GET /wallet/`	balance, `scan_count`, `chat_count`	farmer
`POST /mpesa/topup/`	initiate Daraja STK push	farmer, `payments` throttle
`POST /mpesa/callback/` *(root)*	Safaricom result callback	IP allowlist

Status: Partial — sandbox credentials work; production cutover is an open roadmap item.

Top-up request: `{amount, phone_number}` (decimal, serializer-bounded). Response: `200` with `{status:"Success", message, checkout_request_id}` or `502` when Daraja rejects the push.

Callback semantics: look up `MpesaTransaction` by `CheckoutRequestID`; `ResultCode == 0` → `status='Success'` and `wallet.balance += amount`; otherwise `status='Failed'`. Always answers `200 {ResultCode: 0}`.

Known gaps:

- The callback credits `mpesa_txn.amount` (the value set at initiation) without confirming against Safaricom, and the IP allowlist is inert when `MPESA_CALLBACK_ALLOWLIST` is empty (default) or `DEV_ENV=True` — a forged callback can credit the wallet.
- Code reads `CONSUMER_KEY/CONSUMER_SECRET` while `infra/.env.example` ships `MPESA_CONSUMER_*`; sandbox shortcode/passkey are hardcoded as fallbacks (`mpesa_utils.py:13-17`).

3.8 F8 — Farm Health Score & lending API

Method & path	Purpose	Access
`GET /lender/me/`	own organisation profile	lender
`PATCH /lender/me/`	update profile	lender
`GET /lender/api-keys/`	list keys (secret shown once)	lender
`POST /lender/api-keys/`	create key	lender
`POST /lender/api-keys/<id>/rotate/`	rotate secret	approved lender
`POST /lender/api-keys/<id>/revoke/`	revoke key	lender
`GET /lender/usage/`	usage log	lender
`GET /lender/score/?phone=`	score via session JWT	approved lender
`GET /lending/score/?phone=`	score via API key (M2M)	Bearer key

Status: Partial — infrastructure complete; the roadmap marks it *"do not sell"*. The score is gameable (repeat healthy scans inflate it) and penalises farmers who scan sick crops — documented in `docs/business_plan_identification.md`.

Score payload: `farmer_name`, `county`, `farm_size_acres`, `crops`, `health_score`, `risk_tier`, `active_connections`, `total_scans`.

Gate conditions:

- Farmer must have `opt_in_for_lending = true` → otherwise `403 "User has not consented to data sharing"`.
- Unknown phone → `404 "Farmer not found"` (phone normalisation tries `07xx` ↔ `2547xx`).
- Missing/invalid API key → `401`.
- Every M2M call is written to `LendingAPIUsageLog` regardless of outcome.
- Key rotation requires `verification_status == "approved"`; creation/reads do not.

3.9 F9 — Admin (moderation & revenue)

Method & path	Purpose	Access
`GET /admin/revenue/`	revenue KPIs + transactions	staff
`GET /admin/lenders/`	list lending partners	staff
`POST /admin/lenders/<id>/approve/`	approve partner	staff
`POST /admin/lenders/<id>/suspend/`	suspend partner	staff

Status: Implemented (API). The Django `/admin/` site is *commented out* in `AgriProject/urls.py`, so moderation happens only through these endpoints (surfaced at `/admin/*` in the web app).

3.10 F10 — USSD channel

Method & path	Purpose	Access
`POST /ussd/`	production webhook (Africa's Talking text)	provider secret header
`POST /ussd/simulate/`	JSON simulator for the homepage demo	anon, `ussd` throttle
`GET /ussd/simulate/`	simulator menu metadata	anon

Status: Partial — simulator live; real provider credentials pending (`USSD_PROVIDER=simulate` default).

Both endpoints run the same menu engine (`accounts/ussd_engine.py`), session state in cache with a 180 s TTL, Africa's Talking-style `text` of `"` → `1` → `1*2`...

Menus: **1** wallet, **2** AgriBot (free text), **3** market, **4** inputs (fertiliser/seeds/pesticides), **5** Farm Health Score, **6** lending consent, **7** profile, **8** top-up help, **0** exit. Scanning is deliberately excluded (camera).

Gateway security: `X-AgriScan-USSD-Secret` compared with `hmac.compare_digest`; fails closed when no secret is configured (outside `DEV_ENV`). `GET /ussd/simulate/` returns the menu feature list plus the documented exclusion (`"crop_scan"`: "Scanning needs a camera — use the AgriScan app").

3.11 F11 — Legacy server-rendered routes

Path (root)	Purpose	Status
`/`	marketing homepage	Legacy
`/privacy-policy`	privacy policy	Legacy
`/login`, `/logout/`	session login/logout (HTML)	Legacy
`/password-reset/...`	4-step Django password reset	Implemented
`/process`, `/chat/`, `/fertiliser`	old HTML scan/chat/forms	Legacy
`/market/`, `/market_firms/`, `/farmers`, `/farmer_data/`	old directories (**unmasked PII**)	Legacy
`/profile/farmer/`, `/dashboard/buyer/`	old portals	Legacy
`/mpesa/topup/`, `/mpesa/callback/`	HTML-side top-up + callback	Partial
`/admin/revenue/`, `/site-admin/login/`	HTML admin	Legacy
`/accounts/...`	django-allauth (Google OAuth)	Implemented
`/media/...`	media files (dev only)	Implemented
`/admin/` (Django admin)	**commented out**	Planned

Note: the JSON API and the legacy HTML views duplicate several features (score, top-up, directories) with *different* auth and masking behaviour; new work should target the JSON API.

4 Access matrix

Endpoint group	Anon	Farmer	Buyer	Agency	Lender
Scans	—	✓	—	—	—
Chat	—	✓	—	—	—
Wallet / top-up	—	✓	—	—	—
Market prices	—	✓	✓	—	—
Buyers dir	—	✓	—	—	—
Farmers dir	—	—	✓	—	—
Inputs browse	—	✓	—	—	—
Inputs CRUD	—	—	—	✓	—
Lender self-serve	—	—	—	—	✓
Admin	—	—	—	—	staff only
Health / USSD sim	✓	✓	✓	✓	✓

(-- = 401/403; ✓= allowed)

Environment matrix

1. **Laptop dev:** `DEV_ENV=True` (throttles off, CORS open, LocMem cache) — the README quick path.
2. **Production-like:** `DEV_ENV=False` behind `docker compose -f docker-compose.yml` + nginx (Redis cache, throttles on, HTTPS). The README's plain `docker compose up` also loads `docker-compose.override.yml`, which re-exposes 5432/6379/8000 and forces `DEBUG=True` — not a production run.

Error-contract conventions

- DRF field errors → `400` with field-keyed dict.
- Domain errors → `400/401/403/404` with `{"detail": "..."}` or `{"error": "..."}` — two different keys are in use.
- Payment required → `402`.
- Backpressure → `429/503` with `Retry-After`.

5 Appendix — Endpoint index

Method	Path	Feature
GET	/health/	F1
GET	/crops/	F3
POST	/scans/	F3
GET	/scans/	F3
GET	/scans/{job_id}/	F3
POST	/auth/register/farmer/	F2
POST	/auth/register/buyer/	F2
POST	/auth/register/agency/	F2
POST	/auth/register/lender/	F2
POST	/auth/login/	F2
POST	/auth/refresh/	F2
POST	/auth/logout/	F2
GET	/me/	F2
PATCH	/me/	F2
GET	/weather/	F2
POST	/chat/messages/	F4
GET	/market/kamis-prices/	F5
GET	/market/buyers/	F5
GET	/market/farmers/	F5
GET	/inputs/agencies/	F6
GET	/inputs/products/	F6
GET	/inputs/my-products/	F6
POST	/inputs/my-products/	F6
PATCH	/inputs/my-products/{id}/	F6
DELETE	/inputs/my-products/{id}/	F6
GET	/wallet/	F7
POST	/mpesa/topup/	F7
POST	/mpesa/callback/ (root)	F7
GET	/lender/me/	F8
PATCH	/lender/me/	F8
GET	/lender/api-keys/	F8
POST	/lender/api-keys/	F8
POST	/lender/api-keys/{id}/rotate/	F8
POST	/lender/api-keys/{id}/revoke/	F8
GET	/lender/usage/	F8
GET	/lender/score/	F8
GET	/lending/score/	F8
GET	/admin/revenue/	F9
GET	/admin/lenders/	F9
POST	/admin/lenders/{id}/approve/	F9
POST	/admin/lenders/{id}/suspend/	F9
POST	/ussd/	F10
POST	/ussd/simulate/	F10
GET	/ussd/simulate/	F10

Generated from the AgriScan monorepo source. Status markers reflect the codebase as of commit `23c9c5b`; re-verify after each merge.